

Developer platform

React web, Express API, and PostgreSQL run as independent Helm releases: `web`, `api`, and `db`. API and web use one shared application chart; PostgreSQL uses a separately pinned upstream chart.

Environments and delivery

Environment	Cluster / namespace	Delivery
Local	testkube-platform / testkube-sample	Build locally and load images into kind
CI	testkube-samples / app-ci	Feature push resets releases and database data; builds images and runs acceptance tests
Development	testkube-samples / app-dev	Main push promotes passing CI images without rebuilding; preserves database data
Production	testkube-samples / app-prod	Manual release publishes artifacts; environment approval gates deployment

Feature push -> app-ci tests -> ci-passed
Merge to main -> verify source and images -> app-dev -> dev-passed
Manual release -> image tags + GitHub Release -> approval -> app-prod

Shared images are stored in public GHCR packages and deployed by digest. CI publishes `ci-<run number>` and `ci-latest`; development uses `dev-latest`; production publishes `prod-YYYYMMDDTHHMMSSZ` and `latest`. Helm APP VERSION shows the CI build in CI/development and the timestamp tag in production. Release notes list merged PRs since the previous published production release. One concurrency group serializes CI, development, and production, including approval waits.

Merge queues

Merge queues are not used because this repository is personally owned, rather than owned by a GitHub organization. [GitHub merge queues require organization ownership](#). The current workflow runs acceptance and app-ci deployment on feature-branch pushes.

With organization ownership, the intended flow would be:

- PR updates: run automated local acceptance checks in a workflow.
- Merge queue entry: handle `merge_group`, deploy the combined candidate to app-ci,

and run deployment acceptance tests before allowing the merge.

The PR workflow would invoke the individual checks; make `local-tests` remains developer-only and is not called by workflows. This merge-queue flow is not implemented.

Ownership and configuration

- Application repo: source, Dockerfiles, `platform.json`, `deploy/values`, acceptance tests, and workflow callers.
- Platform repo: CLI, lifecycle scripts, shared chart, tool versions, and reusable workflows.

- `platform.lock.json` pins `platform-tools v0.8.0` and its manifest checksum; workflow references use the same tag.
- GitHub environments supply `KUBECONFIG`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`.

Each namespace has its own deployment identity and database Secret shared by API and PostgreSQL.

Developer and operator commands

```
make setup          # Install pinned CLI
make local-tests    # Full local acceptance; leaves cluster running
make open           # Web :4173 and API :8080 tunnels
make deploy SERVICE=web
make check
make status
make releases
make rollback SERVICE=web REVISION=<number>
make down CONFIRM=testkube-platform # Delete local cluster and database data
```

CI runs chart, API, frontend build, HTTP, release isolation, and browser checks. Development and production run rollout and HTTP checks. Local acceptance also runs the database persistence/outage drill. Application rollback restores one Helm revision; it does not restore database data or external Secrets.

Operational boundaries

Shared environments have separate namespaces and RBAC but share cluster failure domains. Database backups/PITR, high availability, enforced network isolation, public TLS routing, and centralized telemetry are not implemented. Browser access requires local tunnels; API readiness does not detect database failure. Moving CI aliases can block promotion when another branch replaces the tested images.

Deferred work

The following items were deferred due to the four-hour implementation budget:

- Automated image cleanup. Images currently remain in GHCR; a retention policy should preserve deployed and rollback images before deleting unused builds.
- A manual workflow to select an existing release, deploy its recorded image

digests to `app-ci`, and run acceptance tests. Current CI builds and tests the pushed branch; it does not support testing an operator-selected release.

- Build only services affected by a PR. For unchanged services, reuse the

corresponding `dev-latest` image digest and add the current `ci-<run number>` tag instead of rebuilding. Changes to shared code or build inputs should also trigger builds of affected services. Compare against the source revision recorded in `dev-latest`, including shared code, dependencies, and Dockerfiles; the PR changed-file list alone is insufficient. Resolve `dev-latest` once, then tag and deploy that exact digest. Build the service if no development image exists. Update promotion checks to accept reused images with their original source and build metadata; adding a tag does not change image labels.

For procedures, use `README.md`, `DEVELOPER-GUIDE.md`, `DEVELOPMENT.md`, and `PRODUCTION.md`.